

LABORATORIO 08

Dejar la libreta ordenada

Guía para leer mientras trabajas el cuaderno. Cada celda trae la pregunta que responde, la idea en el almacén, la sentencia explicada parte por parte, lo que sale en pantalla y cómo leerlo.

Introducción

1 · El tema

Este es el laboratorio más operativo. Vas a hacer la rutina de mantención que una tabla Iceberg necesita cuando recibe datos todos los días. Juntar los archivos chicos, botar la historia que ya no se va a pedir y buscar archivos que nadie reclama.

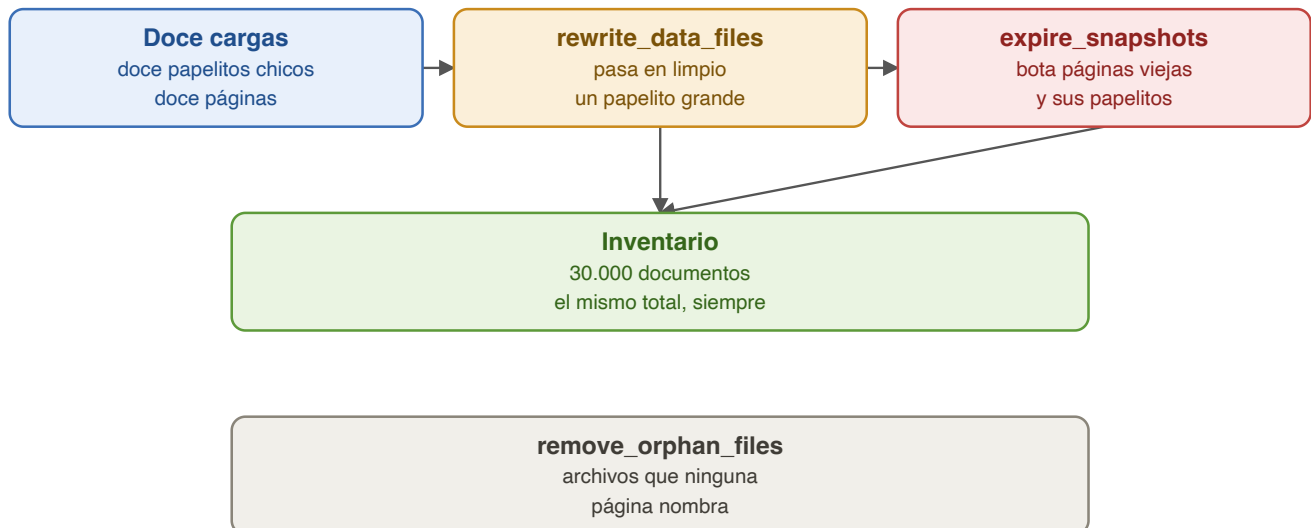
2 · El problema, en el almacén

Al almacenero le llegan pedidos a toda hora, y cada uno lo anota en un papelito suelto. Al final del mes la información está completa, pero para encontrar el pedido de un cliente hay que revisar un montón de papelitos. Y la libreta lleva años de páginas viejas en un cajón que no es infinito.

3 · El problema en términos técnicos

Cada carga escribe al menos un archivo Parquet nuevo. Una tabla que recibe lotes chicos todos los días termina con cientos o miles de archivos diminutos, y cada archivo que una consulta tiene que abrir se paga. Hay que ir a buscarlo, abrirlo y leer su pie antes de mirar una sola fila. Además, cada carga deja una página, y las páginas viejas mantienen vivos los archivos viejos, así que el espacio en disco no baja solo.

4 · El diagrama



Doce cargas (azul). La tabla se arma con una carga por mes, como se arma sola una tabla que recibe datos seguido. Cada carga deja un papelito chico y una página.

rewrite_data_files (amarillo). Compacta. Lee los papelitos chicos y escribe uno grande con el mismo contenido. No borra los viejos, porque las páginas anteriores los siguen nombrando.

expire_snapshots (rojo). Bota las páginas anteriores a una fecha de corte, y con ellas los papelitos que solo esas páginas usaban. No se puede deshacer.

Inventario (verde). Antes y después de cada rutina se cuentan los documentos y se suma el total. Tiene que dar siempre lo mismo, porque la mantención no toca los datos vigentes.

remove_orphan_files (gris). Busca archivos que quedaron en la carpeta sin que ninguna página los nombre, por ejemplo de una escritura que se cortó. Es la rutina más delicada, y se corre primero sin borrar.

5 · La solución

En el almacén, el almacenero pasa en limpio los papelitos a una hoja ordenada, con la misma información en un solo papel. Guarda los borradores mientras alguien pueda pedirlos, y cuando se decide que ya nadie los va a pedir, los bota. De vez en cuando revisa si quedaron papeles sueltos que no pertenecen a ninguna página.

En Iceberg eso son tres procedimientos. `rewrite_data_files` compacta, `expire_snapshots` bota la historia anterior a un corte y `remove_orphan_files` busca huérfanos. Los tres se llaman con `CALL`, y se programan como cualquier proceso batch.

6 · Los pasos

- **Paso 0.** Armas la tabla `documentos_recibidos` con doce cargas, una por mes, y levantas el inventario.
- **Paso 1.** Mides el costo, doce archivos chicos para 30000 documentos.
- **Paso 2.** Compactas y compruebas que los datos no cambiaron.
- **Paso 3.** Ves que compactar no liberó espacio y por qué.
- **Paso 4.** Botas la historia vieja con `expire_snapshots` y ves que es irreversible.
- **Paso 5.** Buscas archivos huérfanos, sin borrar.
- **Paso 6.** La rutina, cada cuánto y en qué orden.
- **Paso 7.** Lo que la mantención ordena, tabla por tabla.

Si ya corriste este laboratorio, la celda 0.2 borra la tabla y se puede volver a correr desde arriba.

Paso 0 · Un año de ingesta, mes a mes

Celda 0.1 · Entrar a tu espacio

La pregunta. ¿En qué espacio va a quedar la tabla?

Por qué ahora. La tabla de hoy se crea en tu espacio de trabajo, `mi_espacio`.

En el almacén. Es pararse frente a tu propio mostrador.

La sentencia, parte por parte.

- `%%sql` le dice al cuaderno que lo que sigue es SQL.
- `-- Celda 0.1` es un comentario que el motor ignora.
- `USE mi_espacio` deja a `mi_espacio` como espacio por omisión.

```
sql
%%sql
-- Celda 0.1
USE mi_espacio
```

Lo que sale en pantalla.

```
text
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/25 18:23:55 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se ejecutó. Las tres primeras líneas son avisos de arranque de Spark, no errores.

- `Setting default log level to "WARN"` dice que Spark va a mostrar solo avisos y errores.
- `To adjust logging level use sc.setLogLevel(newLevel)` explica cómo cambiar ese nivel, con `sc`, el contexto de Spark. SparkR es la versión de Spark para el lenguaje R, que aquí no se usa.
- `WARN NativeCodeLoader: Unable to load native-hadoop library` dice que no encontró una biblioteca de Hadoop compilada para este sistema y usa la de Java. `26/09/25 18:23:55` es la fecha y la hora, 25 de septiembre de 2026 a las 18:23:55 UTC.

Celda 0.2 · Partir de cero

La pregunta. ¿Cómo se asegura que la tabla parta vacía?

Por qué ahora. Si la tabla ya existe, crearla falla.

En el almacén. Es sacar la libreta vieja del mostrador.

La sentencia, parte por parte.

- `DROP TABLE` borra la tabla.
- `IF EXISTS` evita el error si no existe.
- `documentos_recibidos` es la tabla, en `mi_espacio`.

```
sql
%%sql
-- Celda 0.2
DROP TABLE IF EXISTS documentos_recibidos
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se ejecutó.

Celda 0.3 · La carga de enero

La pregunta. ¿Cómo se crea la tabla con los documentos de enero?

Por qué ahora. Es la primera carga del año. La tabla nace con ella.

En el almacén. Es abrir la libreta y anotar el primer papelito, el de enero.

La sentencia, parte por parte.

- `CREATE TABLE documentos_recibidos USING iceberg` crea una tabla Iceberg.
- `AS SELECT * FROM curso.dte_2024` la llena con los documentos de 2024 que trae el ambiente en el espacio `curso`.
- `WHERE month(fecha_emision) = 1` se queda solo con enero. `month` saca el número del mes de una fecha.

```
sql
%%sql
-- Celda 0.3
CREATE TABLE documentos_recibidos USING iceberg AS
SELECT * FROM curso.dte_2024
WHERE month(fecha_emision) = 1
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La tabla quedó creada con enero. Es un papelito y una página.

Celda 0.4 · La carga de febrero

La pregunta. ¿Cómo se agregan los documentos de febrero?

Por qué ahora. Es la carga número 2 del año. Desde aquí la tabla ya existe, así que no es `CREATE` sino `INSERT INTO`. Cada carga deja su propio papelito y su propia página.

En el almacén. Es anotar el papelito de febrero.

La sentencia, parte por parte.

- `INSERT INTO documentos_recibidos` agrega filas a la tabla.
- `SELECT * FROM curso.dte_2024` trae los documentos de 2024.
- `WHERE month(fecha_emision) = 2` se queda solo con febrero, el mes 2.

```
sql
%%sql
-- Celda 0.4
INSERT INTO documentos_recibidos
SELECT * FROM curso.dte_2024
WHERE month(fecha_emision) = 2
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Entró febrero. La tabla suma un papelito y una página más.

Celda 0.5 · La carga de marzo

La pregunta. ¿Cómo se agregan los documentos de marzo?

Por qué ahora. Es la carga número 3 del año. Cada carga deja su propio papelito y su propia página.

En el almacén. Es anotar el papelito de marzo.

La sentencia, parte por parte.

- `INSERT INTO documentos_recibidos` agrega filas a la tabla.
- `SELECT * FROM curso.dte_2024` trae los documentos de 2024.
- `WHERE month(fecha_emision) = 3` se queda solo con marzo, el mes 3.

```
sql
%%sql
-- Celda 0.5
INSERT INTO documentos_recibidos
SELECT * FROM curso.dte_2024
WHERE month(fecha_emision) = 3
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Entró marzo. La tabla suma un papelito y una página más.

Celda 0.6 · La carga de abril

La pregunta. ¿Cómo se agregan los documentos de abril?

Por qué ahora. Es la carga número 4 del año. Cada carga deja su propio papelito y su propia página.

En el almacén. Es anotar el papelito de abril.

La sentencia, parte por parte.

- `INSERT INTO documentos_recibidos` agrega filas a la tabla.
- `SELECT * FROM curso.dte_2024` trae los documentos de 2024.
- `WHERE month(fecha_emision) = 4` se queda solo con abril, el mes 4.

```
sql
%%sql
-- Celda 0.6
INSERT INTO documentos_recibidos
SELECT * FROM curso.dte_2024
WHERE month(fecha_emision) = 4
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Entró abril. La tabla suma un papelito y una página más.

Celda 0.7 · La carga de mayo

La pregunta. ¿Cómo se agregan los documentos de mayo?

Por qué ahora. Es la carga número 5 del año. Cada carga deja su propio papelito y su propia página.

En el almacén. Es anotar el papelito de mayo.

La sentencia, parte por parte.

- `INSERT INTO documentos_recibidos` agrega filas a la tabla.
- `SELECT * FROM curso.dte_2024` trae los documentos de 2024.
- `WHERE month(fecha_emision) = 5` se queda solo con mayo, el mes 5.

```
sql
%%sql
-- Celda 0.7
INSERT INTO documentos_recibidos
SELECT * FROM curso.dte_2024
WHERE month(fecha_emision) = 5
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Entró mayo. La tabla suma un papelito y una página más.

Celda 0.8 · La carga de junio

La pregunta. ¿Cómo se agregan los documentos de junio?

Por qué ahora. Es la carga número 6 del año. Cada carga deja su propio papelito y su propia página.

En el almacén. Es anotar el papelito de junio.

La sentencia, parte por parte.

- `INSERT INTO documentos_recibidos` agrega filas a la tabla.
- `SELECT * FROM curso.dte_2024` trae los documentos de 2024.
- `WHERE month(fecha_emision) = 6` se queda solo con junio, el mes 6.

```
sql
%%sql
-- Celda 0.8
INSERT INTO documentos_recibidos
SELECT * FROM curso.dte_2024
WHERE month(fecha_emision) = 6
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Entró junio. La tabla suma un papelito y una página más.

Celda 0.9 · La carga de julio

La pregunta. ¿Cómo se agregan los documentos de julio?

Por qué ahora. Es la carga número 7 del año. Cada carga deja su propio papelito y su propia página.

En el almacén. Es anotar el papelito de julio.

La sentencia, parte por parte.

- `INSERT INTO documentos_recibidos` agrega filas a la tabla.
- `SELECT * FROM curso.dte_2024` trae los documentos de 2024.
- `WHERE month(fecha_emision) = 7` se queda solo con julio, el mes 7.

```
sql
%%sql
-- Celda 0.9
INSERT INTO documentos_recibidos
SELECT * FROM curso.dte_2024
WHERE month(fecha_emision) = 7
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```


Cómo se lee. Entró julio. La tabla suma un papelito y una página más.

Celda 0.10 · La carga de agosto

La pregunta. ¿Cómo se agregan los documentos de agosto?

Por qué ahora. Es la carga número 8 del año. Cada carga deja su propio papelito y su propia página.

En el almacén. Es anotar el papelito de agosto.

La sentencia, parte por parte.

- `INSERT INTO documentos_recibidos` agrega filas a la tabla.
- `SELECT * FROM curso.dte_2024` trae los documentos de 2024.
- `WHERE month(fecha_emision) = 8` se queda solo con agosto, el mes 8.

```
sql
%%sql
-- Celda 0.10
INSERT INTO documentos_recibidos
SELECT * FROM curso.dte_2024
WHERE month(fecha_emision) = 8
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Entró agosto. La tabla suma un papelito y una página más.

Celda 0.11 · La carga de septiembre

La pregunta. ¿Cómo se agregan los documentos de septiembre?

Por qué ahora. Es la carga número 9 del año. Cada carga deja su propio papelito y su propia página.

En el almacén. Es anotar el papelito de septiembre.

La sentencia, parte por parte.

- `INSERT INTO documentos_recibidos` agrega filas a la tabla.
- `SELECT * FROM curso.dte_2024` trae los documentos de 2024.
- `WHERE month(fecha_emision) = 9` se queda solo con septiembre, el mes 9.

```
sql
%%sql
-- Celda 0.11
INSERT INTO documentos_recibidos
SELECT * FROM curso.dte_2024
WHERE month(fecha_emision) = 9
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Entró septiembre. La tabla suma un papelito y una página más.

Celda 0.12 · La carga de octubre

La pregunta. ¿Cómo se agregan los documentos de octubre?

Por qué ahora. Es la carga número 10 del año. Cada carga deja su propio papelito y su propia página.

En el almacén. Es anotar el papelito de octubre.

La sentencia, parte por parte.

- `INSERT INTO documentos_recibidos` agrega filas a la tabla.
- `SELECT * FROM curso.dte_2024` trae los documentos de 2024.
- `WHERE month(fecha_emision) = 10` se queda solo con octubre, el mes 10.

```
sql
%%sql
-- Celda 0.12
INSERT INTO documentos_recibidos
SELECT * FROM curso.dte_2024
WHERE month(fecha_emision) = 10
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Entró octubre. La tabla suma un papelito y una página más.

Celda 0.13 · La carga de noviembre

La pregunta. ¿Cómo se agregan los documentos de noviembre?

Por qué ahora. Es la carga número 11 del año. Cada carga deja su propio papelito y su propia página.

En el almacén. Es anotar el papelito de noviembre.

La sentencia, parte por parte.

- `INSERT INTO documentos_recibidos` agrega filas a la tabla.
- `SELECT * FROM curso.dte_2024` trae los documentos de 2024.
- `WHERE month(fecha_emision) = 11` se queda solo con noviembre, el mes 11.

```
sql
%%sql
-- Celda 0.13
INSERT INTO documentos_recibidos
SELECT * FROM curso.dte_2024
WHERE month(fecha_emision) = 11
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Entró noviembre. La tabla suma un papelito y una página más.

Celda 0.14 · La carga de diciembre

La pregunta. ¿Cómo se agregan los documentos de diciembre?

Por qué ahora. Es la carga número 12 del año. Cada carga deja su propio papelito y su propia página.

En el almacén. Es anotar el papelito de diciembre.

La sentencia, parte por parte.

- `INSERT INTO documentos_recibidos` agrega filas a la tabla.
- `SELECT * FROM curso.dte_2024` trae los documentos de 2024.
- `WHERE month(fecha_emision) = 12` se queda solo con diciembre, el mes 12.

```
sql
%%sql
-- Celda 0.14
INSERT INTO documentos_recibidos
SELECT * FROM curso.dte_2024
WHERE month(fecha_emision) = 12
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Entró diciembre. La tabla suma un papelito y una página más.

Celda 0.15 · El inventario

La pregunta. ¿Cuántos documentos quedaron y cuánto suman?

Por qué ahora. Esta medición se va a repetir dos veces más, para comprobar que la mantención no toca los datos.

En el almacén. Es contar las facturas y sumar sus montos antes de ordenar nada.

La sentencia, parte por parte.

- `count(*) AS documentos` cuenta las filas.
- `sum(monto_total) AS total_general` suma los montos.
- `FROM documentos_recibidos` es la tabla.

```
sql
%%sql
-- Celda 0.15
SELECT count(*)          AS documentos,
       sum(monto_total) AS total_general
FROM documentos_recibidos
```

Lo que sale en pantalla.

```
text
documentos  total_general
30000      291293351462.71
1 fila.
```

Cómo se lee. 30000 documentos que suman 291.293.351.462,71 pesos. Anota ese total.

Paso 1 · Medir el costo

Celda 1.1 · Archivos y tamaños

La pregunta. ¿En cuántos archivos está guardada la tabla y cuánto pesan?

Por qué ahora. Es la medición del problema.

En el almacén. Es contar los papelitos y pesarlos.

La sentencia, parte por parte.

- `count(*)` AS `archivos` cuenta los papelitos, porque en `.files` cada fila es un archivo.
- `sum(record_count)` AS `documentos` suma las filas de cada papelito. `record_count` es cuántas filas trae uno.
- `cast(avg(file_size_in_bytes) AS BIGINT)` AS `bytes_por_archivo` saca el promedio de tamaño con `avg` y lo convierte en entero con `cast ... AS BIGINT`. `file_size_in_bytes` es el tamaño de cada archivo.
- `sum(file_size_in_bytes)` AS `bytes_totales` suma los tamaños.
- `FROM mi_espacio.documentos_recibidos.files` es la vista de los papelitos vigentes. Se nombra con tres partes, espacio, tabla y vista.

```
sql
%%sql
-- Celda 1.1
SELECT count(*)                AS archivos,
       sum(record_count)        AS documentos,
       cast(avg(file_size_in_bytes) AS BIGINT) AS bytes_por_archivo,
       sum(file_size_in_bytes)  AS bytes_totales
FROM mi_espacio.documentos_recibidos.files
```

Lo que sale en pantalla.

```
text
archivos  documentos  bytes_por_archivo  bytes_totales
12        30000      74031             888378
1 fila.
```

Cómo se lee. 12 archivos para 30000 documentos, uno por carga. En promedio pesan 74031 bytes, unos 0,07 MB, y en total 888378 bytes, 0,89 MB. Un MB aquí es un millón de bytes.

Iceberg apunta a archivos de 536.870.912 bytes, unos 537 MB, cuando escribe, que es su tamaño objetivo de fábrica, y estos son unas 7.250 veces más chicos. Una tabla que recibe un lote al día junta cientos de archivos así al año.

Celda 1.2 · Uno por uno

La pregunta. ¿Qué trae cada archivo?

Por qué ahora. Para ver que cada archivo es la carga de un mes.

En el almacén. Es mirar el rótulo de cada papelito, desde qué fecha hasta qué fecha trae.

La sentencia, parte por parte.

- `record_count AS documentos` es cuántas filas trae el archivo.
- `file_size_in_bytes AS bytes` es su tamaño.
- `readable_metrics.fecha_emision.lower_bound AS desde` y `upper_bound AS hasta` son el rótulo, la fecha más chica y la más grande del archivo. `readable_metrics` es la columna de `.files` que guarda esos mínimos y máximos en forma legible.
- `FROM mi_espacio.documentos_recibidos.files` es la vista de papelitos.
- `ORDER BY desde` ordena por fecha.

```
sql
%%sql
-- Celda 1.2
SELECT record_count           AS documentos,
       file_size_in_bytes     AS bytes,
       readable_metrics.fecha_emision.lower_bound AS desde,
       readable_metrics.fecha_emision.upper_bound AS hasta
FROM mi_espacio.documentos_recibidos.files
ORDER BY desde
```

Lo que sale en pantalla.

```
text
documentos  bytes  desde      hasta
2500        73189  2024-01-01 2024-01-31
2500        73627  2024-02-01 2024-02-29
2500        73442  2024-03-01 2024-03-31
2500        74229  2024-04-01 2024-04-30
2500        73711  2024-05-01 2024-05-31
2500        74192  2024-06-01 2024-06-30
2500        74452  2024-07-01 2024-07-31
2500        74327  2024-08-01 2024-08-31
2500        74232  2024-09-01 2024-09-30
2500        74333  2024-10-01 2024-10-31
2500        74467  2024-11-01 2024-11-30
2500        74177  2024-12-01 2024-12-31
12 filas.
```

Cómo se lee. Doce archivos, 2500 documentos cada uno, y cada rótulo cubre un mes exacto. Con esos rótulos, una consulta por junio puede saltarse los otros once. El problema de hoy es otro, lo que cuesta cada archivo que sí hay que abrir.

Celda 1.3 · La consulta de cierre de año

La pregunta. ¿Cuánto suma cada mes?

Por qué ahora. Es una consulta que tiene que abrir todos los archivos.

En el almacén. Es sumar mes por mes revisando todos los papelitos.

La sentencia, parte por parte.

- `date_format(fecha_emision, 'yyyy-MM')` AS periodo convierte la fecha en año y mes.
- `count(*)` AS documentos cuenta y `sum(monto_total)` AS total_del_periodo suma.
- `FROM documentos_recibidos` es la tabla.
- `GROUP BY date_format(fecha_emision, 'yyyy-MM')` agrupa por mes y `ORDER BY periodo` ordena.

```
sql
%%sql
-- Celda 1.3
SELECT date_format(fecha_emision, 'yyyy-MM') AS periodo,
       count(*)                               AS documentos,
       sum(monto_total)                       AS total_del_periodo
FROM documentos_recibidos
GROUP BY date_format(fecha_emision, 'yyyy-MM')
ORDER BY periodo
```

Lo que sale en pantalla.

```
text
periodo  documentos  total_del_periodo
2024-01  2500         23381729910.07
2024-02  2500         24831948613.59
2024-03  2500         26699931432.64
2024-04  2500         23722997990.55
2024-05  2500         24328007216.84
2024-06  2500         24547341762.59
2024-07  2500         23621609483.83
2024-08  2500         23648615665.89
2024-09  2500         24136964210.51
2024-10  2500         23106058024.78
2024-11  2500         24722608269.83
2024-12  2500         24545538881.59
12 filas.
```

Cómo se lee. Doce meses con 2500 documentos cada uno. Para responder hubo que abrir los doce archivos. Con doce no se nota. Con diez mil, el motor pasa más tiempo abriendo archivos que leyendo.

Paso 2 · Pasar en limpio

Celda 2.1 · Compactar

La pregunta. ¿Cómo se juntan los archivos chicos en uno grande?

Por qué ahora. Ya se midió el problema. Esta es la cura.

En el almacén. Es pasar en limpio los papelitos a una hoja ordenada.



12 papelitos (azul). Los archivos de las doce cargas.

rewrite_data_files (amarillo). Los lee y escribe archivos grandes con el mismo contenido. No cambia ni un dato.

1 papelito (verde). El resultado, un archivo con los 30000 documentos.

Los 12 viejos (gris). No se borran. Las páginas anteriores los siguen nombrando, y mientras existan se puede pedir la tabla como estaba antes.

La sentencia, parte por parte.

- `CALL` ejecuta un procedimiento.
- `spark_catalog.system.rewrite_data_files` es el procedimiento de Iceberg que compacta. `spark_catalog` es el catálogo y `system` el espacio de los procedimientos.
- `table => 'mi_espacio.documentos_recibidos'` le pasa la tabla por nombre, entre comillas y con el espacio adelante. `=>` asigna un valor a un argumento con nombre.

```
sql
%%sql
-- Celda 2.1
CALL spark_catalog.system.rewrite_data_files(table => 'mi_espacio.documentos_recibidos')
```

Lo que sale en pantalla.

```
text
rewritten_data_files_count  added_data_files_count  rewritten_bytes_count
12                        1                        888378
1 fila.
```


Cómo se lee. `rewritten_data_files_count` dice que leyó 12 archivos, `added_data_files_count` que escribió 1, y `rewritten_bytes_count` que movió 888378 bytes.

Celda 2.2 · Los archivos, otra vez

La pregunta. ¿Cómo quedaron los archivos después de compactar?

Por qué ahora. Es la misma medición de la celda 1.1, para comparar.

En el almacén. Es contar y pesar de nuevo.

La sentencia, parte por parte. Es la misma de la celda 1.1.

- `count(*)` AS `archivos` cuenta los papelitos vigentes.
- `sum(record_count)` AS `documentos` suma sus filas.
- `cast(avg(file_size_in_bytes) AS BIGINT)` AS `bytes_por_archivo` es el tamaño promedio en entero.
- `sum(file_size_in_bytes)` AS `bytes_totales` es el total.
- `FROM mi_espacio.documentos_recibidos.files` es la vista de papelitos vigentes.

```
sql
%%sql
-- Celda 2.2
SELECT count(*)                AS archivos,
       sum(record_count)        AS documentos,
       cast(avg(file_size_in_bytes) AS BIGINT) AS bytes_por_archivo,
       sum(file_size_in_bytes)  AS bytes_totales
FROM mi_espacio.documentos_recibidos.files
```

Lo que sale en pantalla.

```
text
archivos  documentos  bytes_por_archivo  bytes_totales
1         30000      761212            761212
1 fila.
```

Cómo se lee. 1 archivo con los 30000 documentos, de 761212 bytes, 0,76 MB. El total pesa menos que los 888378 bytes de antes, porque 30000 documentos juntos se comprimen mejor que repartidos en doce, y cada archivito arrastraba su propia cabecera.

Celda 2.3 · El inventario, otra vez

La pregunta. ¿Los datos siguen iguales después de compactar?

Por qué ahora. Compactar no debería cambiar nada. Hay que comprobarlo.

En el almacén. Es contar y sumar después de pasar en limpio.

La sentencia, parte por parte. Es la misma de la celda 0.15.

- `count(*)` AS `documentos` cuenta.
- `sum(monto_total)` AS `total_general` suma.
- `FROM documentos_recibidos` es la tabla.

```
sql
%%sql
-- Celda 2.3
SELECT count(*)          AS documentos,
       sum(monto_total) AS total_general
FROM documentos_recibidos
```

Lo que sale en pantalla.

```
text
documentos  total_general
30000       291293351462.71
1 fila.
```

Cómo se lee. Los mismos 30000 documentos y el mismo total, al centavo. Compactar reorganiza, no altera.

Paso 3 · Lo que no se liberó

Celda 3.1 · Las páginas

La pregunta. ¿Qué páginas tiene la libreta después de compactar?

Por qué ahora. Para ver que la mantención también deja su huella.

En el almacén. Es mirar la lista de páginas fechadas.

La sentencia, parte por parte.

- `SELECT snapshot_id, committed_at, operation` pide el número de página, la hora en UTC y la operación.
- `FROM mi_espacio.documentos_recibidos.snapshots` es la vista de páginas.
- `ORDER BY committed_at` ordena de la más vieja a la más nueva.

```
sql
%%sql
-- Celda 3.1
SELECT snapshot_id, committed_at, operation
FROM mi_espacio.documentos_recibidos.snapshots
ORDER BY committed_at
```

Lo que sale en pantalla.

```
text
snapshot_id      committed_at      operation
7561399047907185895  2026-09-25 18:24:05.778  append
5013964436874795132  2026-09-25 18:24:06.939  append
5605958553486786000  2026-09-25 18:24:08.367  append
1802040594104167461  2026-09-25 18:24:09.361  append
1009743359021141391  2026-09-25 18:24:10.738  append
2700393237366741854  2026-09-25 18:24:12.531  append
4370442779747130215  2026-09-25 18:24:13.891  append
3256189979838852168  2026-09-25 18:24:15.136  append
1041927787630357096  2026-09-25 18:24:16.432  append
17116443147193272    2026-09-25 18:24:17.391  append
5355173705741653654  2026-09-25 18:24:18.644  append
5991328770865681656  2026-09-25 18:24:19.063  append
7875847062182413290  2026-09-25 18:24:24.762  replace
13 filas.
```

Cómo se lee. 13 páginas. Doce `append`, una por carga, y una `replace`, que es la compactación. `replace` significa que se reemplazaron archivos sin cambiar los datos.

La primera, `7561399047907185895`, es la carga de enero, de las 18:24:05 UTC, las 15:24 en Chile. La última, `7875847062182413290`, es la compactación, de las 18:24:24.762 UTC. La décima tiene un número más corto, `17116443147193272`. Los números de página son al azar y a veces salen con menos dígitos.

Celda 3.2 · Viajar a enero

La pregunta. ¿Se puede ver la tabla como estaba después de cargar enero, aunque ya se compactó?

Por qué ahora. Para entender por qué compactar no liberó espacio.

En el almacén. Es abrir la libreta en la página de enero.

La sentencia, parte por parte.

- `count(*) AS documentos` cuenta.
- `FROM documentos_recibidos VERSION AS OF` lee la tabla en una página anterior.
- `<TU_SNAPSHOT_ID>` es un marcador. En su lugar escribes, sin comillas, el `snapshot_id` de la primera fila de la celda 3.1. En la solución fue `7561399047907185895`. Guárdalo, porque se usa otra vez en la celda 4.2.

```
sql
%%sql
-- Celda 3.2
SELECT count(*) AS documentos FROM documentos_recibidos VERSION AS OF <TU_SNAPSHOT_ID>
```

Lo que sale en pantalla.

```
text
documentos
2500
1 fila.
```

Cómo se lee. 2500 documentos, la tabla después de enero. El viaje en el tiempo sigue funcionando, y para eso la tabla tiene que conservar los archivos viejos.

Celda 3.3 · Todo lo que se guarda

La pregunta. ¿Cuántos archivos guarda la tabla, contando los de las páginas viejas?

Por qué ahora. Para ver cuánto espacio ocupa de verdad.

En el almacén. Es contar todos los papeles del cajón, los de la hoja en limpio y los borradores.

La sentencia, parte por parte.

- `count(*) AS archivos_guardados` cuenta los archivos.
- `sum(file_size_in_bytes) AS bytes_ocupados` suma sus tamaños.
- `FROM mi_espacio.documentos_recibidos.all_data_files` es la vista con los archivos de datos de todas las páginas, las vigentes y las viejas.

```
sql
%%sql
-- Celda 3.3
SELECT count(*) AS archivos_guardados,
       sum(file_size_in_bytes) AS bytes_ocupados
FROM mi_espacio.documentos_recibidos.all_data_files
```

Lo que sale en pantalla.

```
text
archivos_guardados  bytes_ocupados
13                  1649590
1 fila.
```

Cómo se lee. 13 archivos que ocupan 1649590 bytes, 1,65 MB. Son el compactado más los doce viejos. La tabla ocupa lo nuevo más lo viejo, y el espacio no se libera solo.

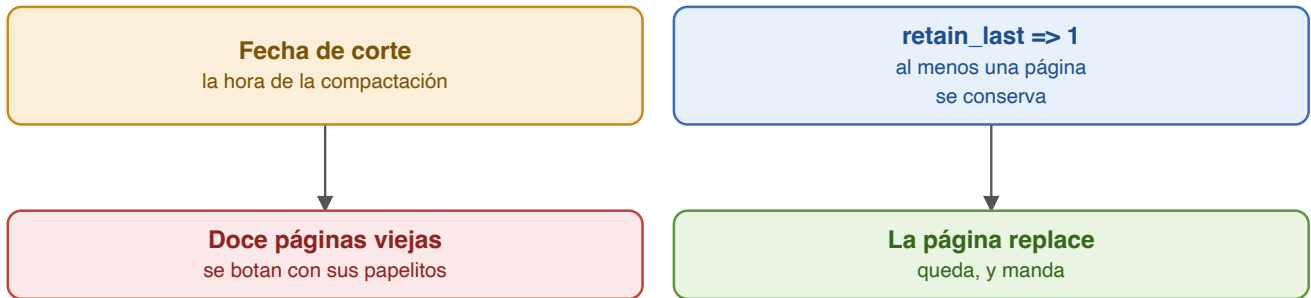
Paso 4 · Botar los borradores

Celda 4.1 · Expirar la historia

La pregunta. ¿Cómo se botan las páginas anteriores a la compactación?

Por qué ahora. Es la única forma de recuperar el espacio. Y es la única operación que destruye información. Después de ella, las páginas anteriores al corte no se pueden consultar y no hay vuelta atrás.

En el almacén. Es botar los borradores que ya nadie va a pedir. Lo que se bota, no vuelve.



Fecha de corte (amarillo). Se bota todo lo anterior a ese instante. Hoy es la hora de la compactación.

retain_last => 1 (azul). Pase lo que pase, se conserva al menos una página.

Doce páginas viejas (rojo). Las de las doce cargas. Se van, y con ellas los papelitos que solo ellas usaban.

La página replace (verde). La de la compactación queda, y es la que manda.

La sentencia, parte por parte.

- `CALL spark_catalog.system.expire_snapshots(` llama al procedimiento que bota páginas.
- `table => 'mi_espacio.documentos_recibidos'` es la tabla.
- `older_than => TIMESTAMP '<TU_ULTIMO_COMMITTED_AT>'` es la fecha de corte. `TIMESTAMP` convierte el texto en fecha y hora. En el marcador escribes, entre las comillas, el `committed_at` de la última fila de la celda 3.1, completo con los milisegundos. En la solución fue `2026-09-25 18:24:24.762`.
- `retain_last => 1` conserva al menos una página.

```
sql
%%sql
-- Celda 4.1
CALL spark_catalog.system.expire_snapshots(
  table      => 'mi_espacio.documentos_recibidos',
  older_than => TIMESTAMP '<TU_ULTIMO_COMMITTED_AT>',
  retain_last => 1)
```

Lo que sale en pantalla.

```
text
deleted_data_files_count  deleted_position_delete_files_count  deleted_equality_delete_files_count  deleted_manifest_files_count  deleted_manifest_lists_count  deleted_statistics_files_count
12                        0                        0                        12                        12                        0
1 fila.
```

Cómo se lee. Es el recibo de lo que se botó. 12 archivos de datos, 12 manifiestos y 12 listas de manifiestos.

- `deleted_data_files_count` son los papelitos borrados, los doce viejos.
- `deleted_position_delete_files_count` y `deleted_equality_delete_files_count` son archivos de borrado, que esta tabla no tiene.
- `deleted_manifest_files_count` son los manifiestos, las listas de papelitos de cada página.
- `deleted_manifest_lists_count` son las listas de manifiestos, una por página.
- `deleted_statistics_files_count` son archivos de estadísticas, que aquí no hay.

Celda 4.2 · Intentar viajar a enero

La pregunta. ¿Se puede ver todavía la página de enero?

Por qué ahora. Es la comprobación de la irreversibilidad. Esta celda falla a propósito.

En el almacén. Es buscar el borrador que se botó.

La sentencia, parte por parte.

- `count(*) AS documentos` cuenta.
- `FROM documentos_recibidos VERSION AS OF <TU_SNAPSHOT_ID>` lee la página anterior. El marcador lleva el mismo número de la celda 3.2, en la solución `7561399047907185895`.

```
sql
%%sql
-- Celda 4.2
SELECT count(*) AS documentos FROM documentos_recibidos VERSION AS OF <TU_SNAPSHOT_ID>
```

Lo que sale en pantalla.

```
text
Spark rechazó la sentencia:

Cannot find snapshot with ID 7561399047907185895
```

Cómo se lee. `Cannot find snapshot with ID 7561399047907185895` dice que esa página no existe. No dice que no encontró datos, dice que la página ya no está. Hace unos minutos respondía 2500, y no hay sentencia que la traiga de vuelta.

Celda 4.3 · Lo que quedó de la historia

La pregunta. ¿Qué páginas quedan?

Por qué ahora. Para ver el efecto en la libreta.

En el almacén. Es mirar la lista de páginas después de botar.

La sentencia, parte por parte.

- `SELECT snapshot_id, committed_at, operation` pide número, hora y operación.
- `FROM mi_espacio.documentos_recibidos.snapshots` es la vista de páginas.
- `ORDER BY committed_at` ordena.

```
sql
%%sql
-- Celda 4.3
SELECT snapshot_id, committed_at, operation
FROM mi_espacio.documentos_recibidos.snapshots
ORDER BY committed_at
```

Lo que sale en pantalla.

```
text
snapshot_id      committed_at      operation
7875847062182413290  2026-09-25 18:24:24.762  replace
1 fila.
```

Cómo se lee. Una sola página, 7875847062182413290, la replace de la compactación. Las doce anteriores se fueron.

Celda 4.4 · El espacio, otra vez

La pregunta. ¿Cuánto ocupa ahora la tabla?

Por qué ahora. Es la misma consulta de la celda 3.3.

En el almacén. Es contar los papeles del cajón después de botar.

La sentencia, parte por parte.

- count(*) AS archivos_guardados cuenta los archivos.
- sum(file_size_in_bytes) AS bytes_ocupados suma los tamaños.
- FROM mi_espacio.documentos_recibidos.all_data_files es la vista de todos los archivos de datos.

```
sql
%%sql
-- Celda 4.4
SELECT count(*) AS archivos_guardados,
       sum(file_size_in_bytes) AS bytes_ocupados
FROM mi_espacio.documentos_recibidos.all_data_files
```

Lo que sale en pantalla.

```
text
archivos_guardados  bytes_ocupados
1                  761212
1 fila.
```


Cómo se lee. 1 archivo de 761212 bytes. De 13 a uno. Ese es el espacio que se recupera.

Celda 4.5 · El inventario, por tercera vez

La pregunta. ¿Los datos vigentes siguen intactos?

Por qué ahora. Se botó historia. Hay que confirmar que no se botaron datos.

En el almacén. Es contar y sumar la hoja en limpio.

La sentencia, parte por parte.

- `count(*)` AS `documentos` cuenta.
- `sum(monto_total)` AS `total_general` suma.
- `FROM documentos_recibidos` es la tabla.

```
sql
%%sql
-- Celda 4.5
SELECT count(*)          AS documentos,
       sum(monto_total) AS total_general
FROM documentos_recibidos
```

Lo que sale en pantalla.

```
text
documentos  total_general
30000       291293351462.71
1 fila.
```

Cómo se lee. 30000 documentos y el mismo total de la celda 0.15. Se botó la historia, no los datos vigentes.

Paso 5 · Los archivos que nadie reclama

Celda 5.1 · Buscar huérfanos, sin borrar

La pregunta. ¿Hay archivos en la carpeta que ninguna página nombra?

Por qué ahora. Cuando una escritura se corta a la mitad pueden quedar archivos escritos que ninguna página menciona. Ocupan espacio y ni compactar ni expirar los ven.

En el almacén. Es buscar papeles sueltos en el cajón que no pertenecen a ninguna página.

La sentencia, parte por parte.

- `CALL spark_catalog.system.remove_orphan_files(` llama al procedimiento que busca y borra huérfanos.
- `table => 'mi_espacio.documentos_recibidos'` es la tabla.
- `dry_run => true` hace que no borre nada y solo liste lo que borraría. Es como se corre siempre la primera vez, porque un archivo que otro proceso está escribiendo todavía no está en ninguna página, y para este procedimiento sería huérfano.

```
sql
%%sql
-- Celda 5.1
CALL spark_catalog.system.remove_orphan_files(
  table    => 'mi_espacio.documentos_recibidos',
  dry_run  => true)
```

Lo que sale en pantalla.

```
text
orphan_file_location
0 filas.
```

Cómo se lee. Ninguna fila. `orphan_file_location` sería la ruta de cada huérfano, y no hay ninguno, porque ninguna escritura se cortó.

Celda 5.2 · La defensa de fábrica

La pregunta. ¿Qué pasa si se piden los huérfanos de la última hora?

Por qué ahora. Para ver la protección que trae Iceberg. Esta celda falla a propósito.

En el almacén. Es querer botar un papel que alguien puede estar escribiendo en este momento.

La sentencia, parte por parte.

- `CALL spark_catalog.system.remove_orphan_files(` llama al procedimiento.
- `table => 'mi_espacio.documentos_recibidos'` es la tabla.
- `older_than => TIMESTAMP '<TU_ULTIMO_COMMITTED_AT>'` pone como corte la misma hora de la celda 4.1, en la solución `2026-09-25 18:24:24.762`, que es de hace minutos.
- `dry_run => true` no borra nada.

```
sql
%%sql
-- Celda 5.2
CALL spark_catalog.system.remove_orphan_files(
  table      => 'mi_espacio.documentos_recibidos',
  older_than => TIMESTAMP '<TU_ULTIMO_COMMITTED_AT>',
  dry_run    => true)
```

Lo que sale en pantalla.

```
text
```

Spark rechazó la sentencia:

Cannot remove orphan files with an interval less than 24 hours. Executing this procedure with a short interval may corrupt the table if other operations are happening at the same time. If you are absolutely confident that no concurrent operations will be affected by removing orphan files with such a short interval, you can use the Action API to remove orphan files with an arbitrary interval.

Cómo se lee. Iceberg se niega. `Cannot remove orphan files with an interval less than 24 hours` dice que no acepta cortes de menos de 24 horas. El resto del mensaje explica por qué. Un intervalo corto puede corromper la tabla si hay otras operaciones en curso, y si estás absolutamente seguro de que no las hay, existe otra vía, la Action API, que es la interfaz de programación de Iceberg. Un archivo recién escrito puede ser de una carga que todavía no confirma, y borrarlo la corrompería.

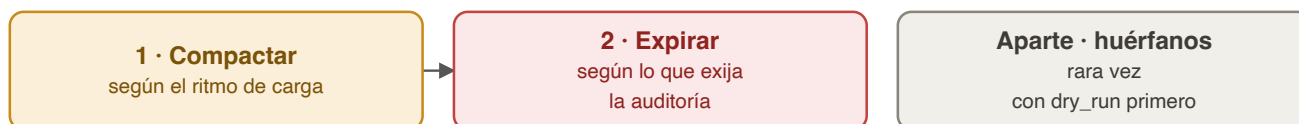
Paso 6 · La rutina

Este paso no tiene celdas. Es texto en el cuaderno, y aquí va explicado.

La pregunta. ¿Cada cuánto y en qué orden se corren las tres rutinas?

Por qué ahora. Ya viste las tres funcionando. Falta el criterio para programarlas.

En el almacén. Pasar en limpio se hace seguido, botar borradores solo cuando se decidió que ya no se piden, y buscar papeles sueltos de vez en cuando y con cuidado.



1 · Compactar (amarillo). Primero, porque es lo que deja historia vieja atrás. Una tabla que recibe un lote al día se compacta una vez por semana, y una que recibe cada hora, todos los días. No cambia datos y se puede repetir.

2 · Expirar (rojo). Después, porque es lo que libera lo que dejó la compactación. Es destructiva, y la fecha de corte es una decisión de auditoría, no un parámetro técnico.

Aparte · huérfanos (gris). Rara vez, siempre con `dry_run` primero y nunca con cargas corriendo. Es la más peligrosa.

Rutina	Qué hace	Cada cuánto	Cuidado
<code>rewrite_data_files</code>	Junta los archivos chicos	Según el ritmo de carga	Ninguno, no cambia datos
<code>expire_snapshots</code>	Bota la historia anterior al corte	Según lo que exija la auditoría	Destructiva
<code>remove_orphan_files</code>	Borra lo que la tabla no reclama	Rara vez, con <code>dry_run</code> primero	Nunca con trabajos escribiendo

Cuánta historia conservar no lo decide el área de datos. Lo decide la exigencia de auditoría de la institución. Si Contraloría puede preguntar por el estado de una tabla a seis meses, la historia se conserva seis meses. Las rutinas no se corren a mano. Se programan con el planificador de trabajos del clúster, en la ventana de menor actividad y nunca con cargas corriendo.

En una tabla particionada, la compactación trabaja dentro de cada cajón. Junta los papelitos de junio entre ellos, sin mezclarlos con julio, y así se ganan archivos grandes sin perder el descarte por cajón.

Paso 7 · Lo que la mantención ordena

Este paso tampoco tiene celdas.

La pregunta. ¿Qué otras cosas de una tabla Iceberg dejan trabajo para estas rutinas?

Por qué ahora. Casi todo lo que se hace en una tabla deja páginas o archivos atrás. Conviene saber cuáles.

En el almacén. Cada anotación, cada corrección y cada libreta aparte deja papeles en el cajón.

- **Cada carga y cada corrección** agregan una página, y las páginas viejas quedan hasta que alguien las expire.
- **Fusionar, corregir y borrar filas** también dejan páginas y papelitos reescritos. Las mismas rutinas los ordenan.
- **Particionar** crea más archivos, uno o más por cajón. Se paga compactando dentro de cada cajón.
- **Eliminar una columna** no reescribe archivos, así que no libera espacio hasta que se compacta y se expira.
- **Una rama viva** protege sus páginas de la expiración. Por eso las ramas de trabajo se botan cuando ya cumplieron.
- **Una tabla migrada desde Hive** puede compartir archivos con su respaldo. Ahí lo que parece huérfano puede ser el dato de otra tabla, y `remove_orphan_files` no es un botón que se aprieta.

Preguntas frecuentes

¿Por qué compactar no liberó espacio?

Porque las páginas anteriores seguían nombrando los archivos viejos. Mientras esas páginas existan, se puede pedir la tabla como estaba, y para eso los archivos tienen que estar. El espacio se libera recién al expirar.

¿Por qué el archivo compactado pesa menos que los doce juntos?

Porque Parquet comprime por bloques, y datos parecidos juntos se comprimen mejor. Además cada archivo chico tenía su propia cabecera y su propio pie.

¿Qué es retain_last?

Cuántas páginas se conservan siempre, aunque sean anteriores a la fecha de corte. Con 1, la tabla nunca queda sin página vigente.

¿Se puede recuperar una página expirada?

No. La página y los archivos que solo ella usaba se borran del disco. Si hace falta conservar un estado, antes hay que ponerle una etiqueta, que lo protege de la expiración.

¿Por qué la fecha va con milisegundos?

Porque la compactación y la última carga ocurrieron con segundos de diferencia. Si la fecha de corte se redondea hacia arriba, se bota también la página que se quería conservar.

¿Qué es un archivo huérfano?

Un archivo que está en la carpeta de la tabla pero que ninguna página nombra. Aparece cuando una escritura se corta antes de confirmar. No lo lee ninguna consulta, pero ocupa espacio.

¿Cuántos archivos chicos son demasiados?

No hay una regla fija. La señal es la que mediste, cantidad de archivos contra tamaño promedio. Cuando el promedio es mucho menor que el tamaño objetivo y los archivos se cuentan por cientos, conviene compactar.

Cierre

La pregunta. ¿Qué te llevas de este laboratorio?

La tabla guarda todo hasta que alguien le dice qué ordenar y qué botar. Compactar ordena sin cambiar datos. Expirar libera espacio y no se deshace. Buscar huérfanos se hace mirando antes de borrar.

En el almacén. Los papelitos se pasan en limpio, los borradores se botan cuando se decidió que ya no se piden, y los papeles sueltos se revisan con cuidado. La información vigente no cambia en ningún momento.

Los números de la solución ejecutada.

Momento	Archivos vigentes	Archivos guardados	Bytes guardados	Páginas	Documentos
Después de las doce cargas	12	12	888378	12	30000
Después de compactar	1	13	1649590	13	30000
Después de expirar	1	1	761212	1	30000

La libreta no se ordena sola. Lo importante es saber qué se pierde si se ordena mal, y por eso la fecha de corte se decide antes de escribir la sentencia.